IFCD107 · ESPECIALISTA EN INTELIGENCIA ARTIFICIAL

Tu primer MLP en Keras

Preparar los datos, Sequential, Dense, compile y fit; leer las curvas y comparar con una referencia sencilla

Sprint 10 · S10_02 · cuaderno: 60 min en clase (ejercicios 1, 2 y 3)

LA IDEA · DEL NUMPY A KERAS

Lo que programaste, Keras lo hace por ti

En S10_01, con NumPy	En Keras
Una capa: `X @ W + b` y `relu`	`Dense(32, activation="relu")`
La red completa: `RedNeuronal`	`keras.Sequential([...])`
`entropia_cruzada`	`loss="binary_crossentropy"`
`paso_gradiente`, con tasa 0,3	`optimizer="adam"`
El bucle de épocas: `entrenar`	`modelo.fit(..., epochs=30)`
`atras`: los gradientes a mano	Automático: TensorFlow los calcula

Keras no cambia la idea: cambia quién escribe el bucle. Lo que no cambia es que tú eliges datos, arquitectura y pérdida.

SECCIÓN 1 · EJERCICIO 1

Las redes son sensibles a la escala

Si **antelacion_dias** va de 0 a 400 y **adultos** de 1 a 4, la primera «grita» más que la segunda.

Qué	Columnas	Transformación
8 numéricas	**antelacion_dias**, **noches**, **adultos**, **ninos**, **peticiones_especiales**, **cliente_repetidor**, **cancelaciones_previas**, **mes_llegada**	**StandardScaler**: media 0, desviación 1
6 categóricas	**tipo_habitacion**, **regimen**, **canal**, **pais_origen**, **tarifa**, **deposito**	**OneHotEncoder**: columnas 0/1
Fuera	**id_reserva**, las fechas en bruto y **cancelada** (el objetivo)	No entran en la red

35

columnas de entrada en clasificación: 9 numéricas (con **precio_noche**) y 26 de one-hot.

Dos detalles

mes_llegada sale de la fecha (la temporada influye).

pais_origen tiene 138 reservas sin país: el codificador las cuenta como una categoría más.

SECCIÓN 1 · SEPARAR LOS DATOS

Entrenamiento, validación y prueba

64 %
Entrenamiento
3 840 reservas: la red ajusta sus pesos

16 %
Validación
960: decidir

20 %
Prueba
1 200: una sola vez

```
A_ent = pre.fit_transform(X_ent)    # solo con entrenamiento
A_val = pre.transform(X_val)
A_pru = pre.transform(X_pru)
```

Fuga de datos

Si el escalador viera la validación o la prueba, «espiaríamos» datos que la red no debe conocer.

La validación sirve para decidir (épocas, arquitectura); la prueba se toca una sola vez, al final.

SECCIÓN 2 · EJERCICIO 2

Dos capas ocultas y una salida

Capa	forma	parámetros
Dense(32, relu)	35 → 32	35 · 32 + 32 = 1 152
Dense(16, relu)	32 → 16	32 · 16 + 16 = 528
Dense(1, sigmoid)	16 → 1	16 + 1 = 17
Total		**1 697**

Parámetros de una capa

entradas · neuronas **+ un sesgo por neurona**. Si el total no cuadra, la forma no es la pedida.

Salida sigmoide + entropía cruzada: la pareja que programaste a mano.

```
modelo = keras.Sequential([keras.layers.Input(shape=(35,))])
modelo.add(keras.layers.Dense(32, activation="relu"))
modelo.add(keras.layers.Dense(16, activation="relu"))
modelo.add(keras.layers.Dense(1, activation="sigmoid"))
modelo.compile(optimizer="adam", loss="binary_crossentropy",
               metrics=[keras.metrics.AUC(name="auc"), "accuracy"])
```

SECCIÓN 2 · ENTRENAR CON FIT

Época, lote y validación

```
hist_c = mlp_c.fit(Xc_ent, yc_ent,
                   validation_data=(Xc_val, yc_val),
                   epochs=30, batch_size=64, verbose=0)
```

Época

Una pasada completa por las **3 840** reservas de entrenamiento.

Lote (batch)

64 filas antes de actualizar los pesos: **60** actualizaciones por época.

validation_data

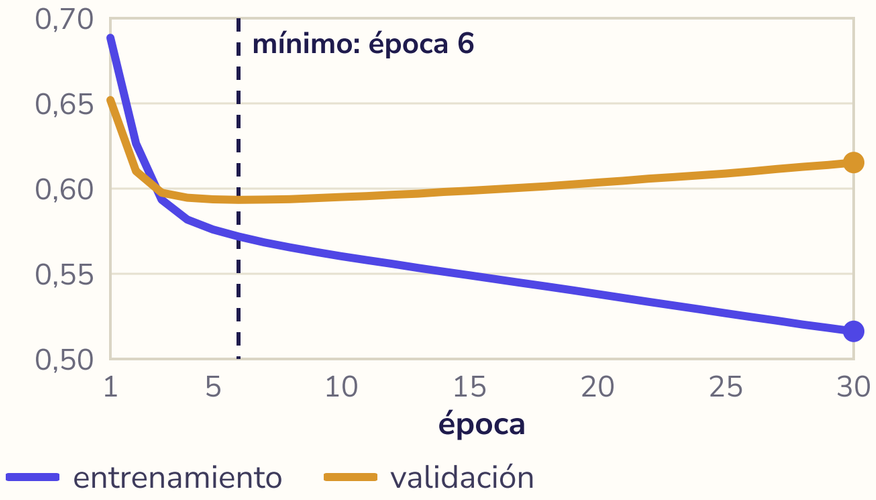
Vigila en cada época con las **960** reservas de validación, **sin entrenar** con ellas.

30 épocas × 60 lotes = **1 800** actualizaciones. **hist_c.history** guarda **loss**, **val_loss**, **auc** y **val_auc** de cada época.

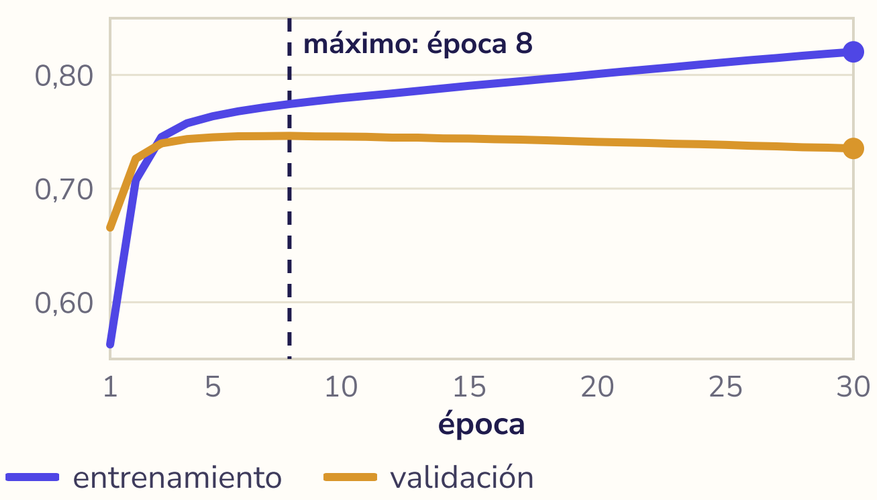
SECCIÓN 2 · CURVAS DE APRENDIZAJE

Lee las curvas: ¿cuándo empieza a memorizar?

Pérdida



AUC



La pérdida de entrenamiento sigue bajando (0,689 → 0,516), pero la de validación toca fondo en la época 6 (0,593) y sube hasta 0,615: sobreajuste.

SECCIÓN 2 · EJERCICIO 3

¿Gana la red a la regresión logística?

Modelo (1 200 reservas de prueba)	AUC	Exactitud
Siempre «no cancela»	0,500	0,541
Regresión logística	**0,746**	0,678
MLP (32-16)	0,727	0,661

Qué pide el ejercicio 3

auc_mlp y **auc_log**: AUC en prueba con **roc_auc_score** sobre las **probabilidades**, no sobre las clases.

mlp_gana: ¿el MLP supera a la logística por más de **0,02** de AUC?

Una cifra sola no dice nada

Acertar un 54 % es lo que consigue quien no sabe nada: la red tiene que superar a un modelo **sencillo**.

Con estos datos mlp_gana es False: la red queda unas 2 centésimas de AUC por debajo de la logística.

PARA CASA · SECCIÓN 3 · EJERCICIOS 4 Y 5

Precio por noche: cambia solo la cabeza

	Clasificación	Regresión
Salida	`Dense(1, "sigmoid")`	`Dense(1)`, sin activación
Pérdida	`binary_crossentropy`	`mse` (métrica legible: `mae`)
Objetivo	0 o 1	precio estandarizado; se deshace para dar euros
Entradas	35 columnas	34: sin `precio_noche`, que es el objetivo

Error medio absoluto en prueba	MAE (€)
Siempre el precio medio	25,52
Regresión lineal (Ridge)	11,87
MLP (32-16)	**11,81**

54 %

menos error que predecir siempre el precio medio (`mejora_vs_media` = 0,537). Ridge y el MLP **empatan**.

PARA CASA · SECCIÓN 4 · MINI-RETO

¿Es mejor la red grande?

Tras 15 épocas	parámetros	AUC entrenamiento	AUC validación	pérdida validación
(32, 16)	1 697	0,791	**0,744**	**0,599**
(128, 64)	12 929	0,849	0,724	0,637

```
mlp_ancho = construir_mlp_clasificacion(n, capas=(128, 64))
hist_ancho = mlp_ancho.fit(..., epochs=15, batch_size=64)
auc_ancho = hist_ancho.history["val_auc"][-1]   # 0.724
```

7,6 veces más parámetros

Más capacidad: **memoriza antes**. El AUC de entrenamiento sube y el de validación baja.

Más parámetros no son un mejor modelo: la red grande no mejora la validación.

HOY HAS APRENDIDO

Keras escribe el bucle, tú decides

Datos para redes

Numéricas escaladas, categóricas en one-hot, escalador ajustado **solo con el entrenamiento**

Sequential → Dense → compile → fit

La salida y la pérdida dependen de la tarea: sigmoide y entropía cruzada, o lineal y MSE

Curvas y referencia

Si la validación sube mientras el entrenamiento baja, hay sobreajuste. Compara siempre con un modelo sencillo

Siguiente: S10_03 · sobreajuste y regularización